Final Report: MealMate

App Overview and Main Features

MealMate is a mobile app designed to help users discover recipes, manage ingredients, and streamline their grocery shopping experience. The app combines recipe discovery with practical shopping cart functionality, creating a complete meal planning ecosystem.

Main Features:

User Authentication System: Email/password registration and login, Firebase authentication integration, persistent login with SharedPreferences, user profile management.

Recipe Discovery: Browse all recipes in the database, search recipes by name or creator email, filter recipes by available ingredients, view "Suggestion of the Day" (most recent recipe), see community-shared recipes on home feed.

Recipe Management: Upload custom recipes with ingredients and cooking steps, edit existing recipes, view detailed cooking instructions.

Ingredient-Based Recipe Finding: Input available ingredients to find matching recipes from the database. Filter results by price, time, and calories.

Smart Shopping Cart: Add individual ingredients or entire recipe ingredient lists, direct links to grocery stores.

User Profile & Content Management: View uploaded recipes count & my uploads, saved recipes section, shopping history. Settings with notification preferences.

Firebase Usage and State Management Approach

Firebase Implementation

Authentication (Firebase Auth):

Uses Firebase Authentication to handle all user-related security operations. The AuthService class manages user sign-up, sign-in, and sign-out operations, stores credentials securely and manages user sessions. The service also handles updating

user display names and provides a stream of authentication state changes that the entire app can listen to.

FirebaseAuth instance provides an authStateChanges stream that notifies the app whenever a user logs in or out. The signIn takes an email and password, calling signInWithEmailAndPassword to authenticate users. The signUp creates new user accounts, and the signOut ends current session.

Database (Cloud Firestore):

The app stores all recipe data in a Firestore collection called "meals". Each recipe document contains comprehensive information including the title, price, time, an array of ingredients, an array of cooking steps, creator identification (ID & email), and a timestamp indicating when it was created.

Real-time data sync is achieved through Firestore streams. The DatabaseService class provides methods that return streams of meal data, ensuring that all connected clients see updates immediately. getMeals returns a stream of all meals ordered by creation date in descending order. Each document snapshot is converted to a Meal object using the fromFirestore factory method.

For user-specific queries, getMealsByUser filters meals by creatorId, allowing users to see only their uploaded recipes. Shopping history is stored in a separate collection called "shopping_history" and follows a similar pattern of real-time updates.

Key Firestore Operations:

The getMeals stream queries the meals collection, orders results by the createdAt timestamp in descending order, listens to snapshots, and maps each document to a Meal object. The getMealsByUser stream adds a where clause to filter by creatorId before performing the same mapping operation.

When adding a new meal, the addMeal method creates a document with all recipe details including title, price, time, creatorId, creatorEmail, ingredients array, cookingSteps array, and a server timestamp. The updateMeal method uses the document ID to update specific fields, while deleteMeal removes the document entirely.

State Management Approach

Provider Pattern:

The app uses the Provider package for state management with three main providers configured in the main app widget. The AuthService provider manages authentication

state globally, making it accessible throughout the widget tree. The DatabaseService provider provides database access to all screens that need it. The StreamProvider for User automatically exposes the current authentication state, rebuilding widgets when the user logs in or out.

In the main.dart file, a MultiProvider wraps the MaterialApp widget, establishing these three providers at the root level. Any widget in the tree can access these providers using context.read or context.watch.

Local State Management:

The CartService uses a singleton pattern to maintain global shopping cart state. This class has a private static instance that ensures only one cart exists throughout the app's lifetime. The factory constructor returns this single instance, making the cart accessible from any screen while maintaining state across navigation.

Individual screens use StatefulWidget for UI-specific state like form inputs, toggles, and temporary selections. SharedPreferences stores persistent user preferences such as notification settings and the last login email, allowing these preferences to survive app restarts.

Real-time Data Flow:

Firestore streams automatically update the UI when database data changes. StreamBuilder widgets listen to these streams and rebuild their child widgets whenever new data arrives. This eliminates the need for manual refresh operations. When one user uploads a recipe, all other users see it immediately without pulling to refresh.

The pattern follows a unidirectional data flow: user actions trigger service methods, which update Firebase, which emits new stream events, which rebuild StreamBuilder widgets. This keeps the UI synchronized with the backend at all times.

Shopping Cart State:

The CartService maintains a Set of ingredient strings, providing add, remove, contains, and clear methods. Since it's a singleton, multiple screens can add items to the same cart. The EmirCShoppingCartScreen reads from this service to display items, while the CookingInstructionsScreen can add entire recipe ingredient lists.

Navigation and Routing:

Named routes defined in main.dart provide a centralized navigation structure. Each route is associated with a builder function that creates the appropriate screen. Some

routes, like the home route, read the current user from the Provider to extract the user's name for personalization.
The Wrapper widget at the root checks the StreamProvider for the current user. If no user is authenticated, it displays the LogIn screen. If a user exists, it navigates to the HomeScreen with the user's name. This automatic routing ensures users always land on the appropriate screen based on authentication state.

Individual Contributions

Teoman Ceyhun Etiz

- main.dart
- signUp.dart
- mainmenu.dart
- auth_service.dart

Rayen Tabassi

- home_screen.dart
- meal_mode_screen.dart
- find_recipe_method_screen.dart
- storage_service.dart

Doruk Kocaman

- profile_cookbook_screen.dart
- upload_recipe_screen.dart
- My_uploads_screen.dart
- ingredient_translator.dart

Emir Ceylan

- shopping_history_screen.dart
- emirc_recipe_info_screen.dart
- emirc_shopping_cart_screen.dart
- cart_service.dart

Mustafa Emir Akcasari

- ingredients_page.dart
- cooking_instructions_screen.dart
- saved_recipes_screen.dart

- database_service.dart

Lessons Learned

- Better understanding of Firebase and real-time data synchronization.
- Better understanding of careful stream management.
- Learned to handle loading and error states in StreamBuilders to prevent crashes.

Challenges Faced

API Link: We tried to link camgoz.net's API to the app, which we got from jojapi.com. Due to several errors and our inexperience with linking APIs in Flutter, we decided to make the app direct you to the grocery store's shopping page instead.

Demonstration Video

https://drive.google.com/file/d/1fROZ-wm29naCVHt-Ref2l7iz_FdRiihR/view?usp=sharing